Expert Systems With Applications 276 (2025) 126957

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Magnitude Attention-based Dynamic Pruning

Jihye Back [a], Namhyuk Ahn [b], Jangho Kim [c,*]

[a] NAVER, Gyeonggi-do, South Korea
[b] Inha University, Incheon, South Korea
[c] Kookmin University, Seoul, South Korea

ARTICLE INFO

Keywords:
Model compression
Model pruning
Dynamic pruning
Deep learning
Optimization

ABSTRACT

Existing pruning methods often rely on weight importance to identify sparse structures but typically apply this information statically, without leveraging it adaptively during training. In this work, we propose a novel approach - **M**agnitude **A**ttention-based Dynamic **P**runing (MAP) method, which applies the importance of weights throughout both the forward and backward paths to explore sparse model structures dynamically. Magnitude attention is defined based on the magnitude of weights as continuous real-valued numbers enabling a seamless transition from a redundant to an effective sparse network by promoting efficient exploration. Additionally, the attention mechanism ensures more effective updates for important layers within the sparse network. Later, our approach shifts from exploration to exploitation, exclusively updating the sparse model composed of crucial weights based on the explored structure, resulting in pruned models that not only achieve performance comparable to dense models but also outperform previous pruning methods on CIFAR-10/100 and ImageNet.

1. Introduction

Deep neural networks (DNNs) have demonstrated exceptional performance across various machine learning domains. However, DNNs are often over-parameterized, resulting in high memory and computational costs during inference (Zhang, Bengio, Hardt, Recht, & Vinyals, 2021). To address this issue, model pruning has been extensively studied, aiming to remove redundant weights or neurons and create sparse models with reduced memory and floating-point operations (FLOPs) compared to dense ones (Chen et al., 2022; Kim, 2023; LeCun, Denker, & Solla, 1989; Singh & Alistarh, 2020; Zhang, Wang, Figueiredo, & Balzano, 2018).

Existing pruning methods typically utilize the importance of weights (or filters) as a static criterion to impose sparsity, focusing on identifying sparse structures rather than dynamically updating weight importance during training (Han, Pool, Tran, & Dally, 2015). Recently, dynamic model pruning methods have emerged, which dynamically update both important and unimportant weights to further explore the sparse structure (Guo, Yao, & Chen, 2016; Lin, Stich, Barba, Dmitriev, & Jaggi, 2020). However, these methods update all weights with the same magnitude throughout the training process, which can lead to performance degradation due to unnecessary updates to unimportant weights. Furthermore, they do not differentiate the importance of weights in the forward and backward propagation during the learning process. Unlike conventional dense model training, where weight importance is not ignored, pruning methods need to incorporate weight importance into their process because all pruning methods select the sparse structure based on their weight importance defined by criteria.

In this work, we propose a novel approach for model pruning, dubbed as **M**agnitude **A**ttention-based dynamic **P**running (MAP), which leverages the magnitude of model weights to identify more important connections in the dense network. Our proposed MAP follows a gradual pruning schedule (Zhu & Gupta, 2017) so the method gradually increases the pruning ratio based on the current iteration (or epoch) until reaching the target pruning ratio (please see Section 3). By considering the on-the-fly element-wise importance of weights in the model, our magnitude attention provides a simple yet effective criterion for identifying important connections without additional costs.

We define the magnitude attention based on the current pruning ratio and the magnitudes of weights. This attention is implemented during both forward and backward paths of the model. In the forward pass, attention is assigned to individual weights based on their importance, guiding the model's updates accordingly. During the backward pass, even non-critical weights are dynamically updated to facilitate a transition to a sparse structure, while also giving more attention to crucial weights for more substantial updates. Furthermore, to promote the exploration and exploitation effect of the sparse model, we introduce two distinct phases. In the *exploration phase*, we apply magnitude

* Corresponding author.
E-mail addresses: 1oojihye@navercorp.com (J. Back), nhahn@inha.ac.kr (N. Ahn), jangho.kim@kookmin.ac.kr (J. Kim).

https://doi.org/10.1016/j.eswa.2025.126957
Received 2 August 2024; Received in revised form 15 January 2025; Accepted 17 February 2025
Available online 10 March 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

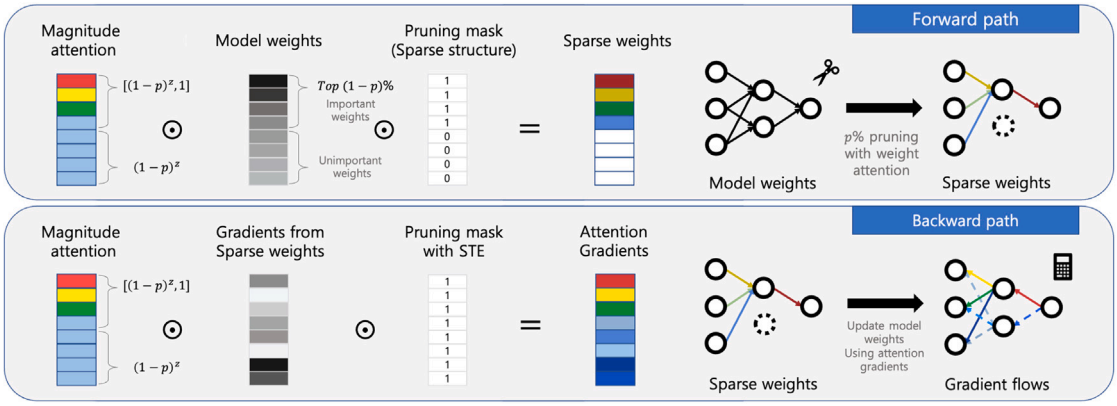

Fig. 1. Magnitude attention mechanism. For clear understanding, the weights of the model are sorted in descending order based on their magnitudes. Unlike conventional pruning methods, MAP multiplies continuous values based on the magnitude of weights in the forward and backward paths. Magnitude attention assigns $(1-p)^z$ to unimportant (pruned) weights, while it assigns values ranging from $(1-p)^z$ to 1 based on the magnitude of important weights, where p is a pruning ratio and z is an attention strength value. During the backward path, the values of the mask corresponding to unimportant weights are set to 1 in order to facilitate exploration via straight-through estimator (Bengio, Léonard, & Courville, 2013).

attention to the search for the sparse structure, while in the *exploitation phase*, we focus on improving the performance and robustness of the discovered sparse structure.

Our algorithm is designed to achieve two main objectives: **(1)** incorporating magnitude attention for model structure exploration, and **(2)** enabling exploitation of the explored sparse structure.

Exploration via Magnitude-Based Attention. During the forward and backward paths, weights are updated based on their importance, with significant weights receiving more attention. Conversely, less important weights receive smaller updates during the backward path. In the early training stages, both important and unimportant weights are updated similarly to facilitate dynamic structure searching (exploration). As the pruning rate increases, the focus shifts to emphasizing updates on important weights while reducing attention to unimportant ones. Our attention mechanism, which enables continuous and differential updates across layers, encourages effective exploration and ensures thorough consideration of each layer (Fig. 1).

Exploitation with Explored Sparse Structure. In the later stages of training, we shift focus from exploration to exploitation. This involves discontinuing the search for potential sparse networks and emphasizing the optimization of the sparse model by maintaining the well-established structure of the discovered sparse networks. Furthermore, we update the model by considering the directions influenced by the layers with the most significant impact on performance (Fig. 2).

The main contributions of our work are as follows:

- We propose a novel magnitude attention-based pruning method (Section 4) and demonstrate its effectiveness by comparing it with competitive state-of-the-art pruning methods (Section 5.2).
- We analyze the impact of our magnitude attention and provide insights on the importance of choosing the appropriate mask or weighting scheme in dynamic pruning (Section 5.3.1).
- We introduce an *exploration–exploitation* strategy for dynamic pruning (Section 4). Notably, we observe that this approach yields highly effective results compared to other static pruning methods.[1]

2. Related work

According to the stage at which the sparsity of a model is imposed for finding a sparse model, pruning methods are mainly classified into three categories: *Sparsify-before-training*, *Sparsify-after-training*, and *Sparsify-during-training*.

Sparsify-after-training is the most basic pruning pipeline, where a dense model is first trained, followed by pruning based on a criterion such as weight magnitude, and then fine-tuning is performed to recover performance. Its simplicity and effective performance recovery make it widely used (Alvarez & Salzmann, 2016; Carreira-Perpinán & Idelbayev, 2018; Han et al., 2015).

Sparsify-before-training searches for the sparse model at an early phase of the training and trains the sparse model with fewer FLOPS compared to a dense model (Hayou, Ton, Doucet, & Teh, 2021; Lee, Ajanthan, & Torr, 2018). Another line of research, the Lottery Ticket Hypothesis (LTH) (Frankle & Carbin, 2018), has demonstrated that using an appropriate initialization based on the sparse structure found through pruning algorithms can maintain the performance achieved in the over-parameterized state.

Sparsify-during-training simultaneously searches for and trains the sparse model from scratch during the training phase, eliminating the need for a re-training phase.

Many of these works rely on the reparameterization of weights through the utilization of either a binary mask or an element-wise nonlinear mapping (Carreira-Perpinan & Idelbayev, 2018; Chen et al., 2023; Ding et al., 2019; Kim, Yoo, & Kwak, 2020; Park, Lee, Mo, & Shin, 2020). Additionally, some studies search for a better sparse structure throughout the entire training process, allowing dead (pruned) weights to be recovered and vice versa, enabling the discovery of a more optimal sparse structure. *Dynamic model pruning* (Guo et al., 2016; Lin et al., 2020) is a type of sparsify-during-training approach that updates even the unimportant weights that do not participate in the forward path.

To achieve this, they utilize a straight-through estimator (STE) (Bengio et al., 2013) for gradient approximation. This method enables the exploration of a more dynamic sparse structure.

Our proposed MAP employs STE, similar to dynamic model pruning, to update unimportant weights. However, in contrast to previous dynamic pruning methods, we introduce magnitude attention and an exploration–exploitation strategy to selectively train weights and boost learning stability.

3. Backgrounds

Dynamic Model Pruning. The pruning process eliminates unimportant weights based on a specified criterion, resulting in a sparse model. Following the methodology introduced by Han et al. (2015), we sort the weights by their magnitudes and assign a zero mask to the smallest

[1] We refer to the approach of updating only the important weights as *static* pruning, which contrasts with the concept of *dynamic* pruning (Guo et al., 2016; Lin et al., 2020).

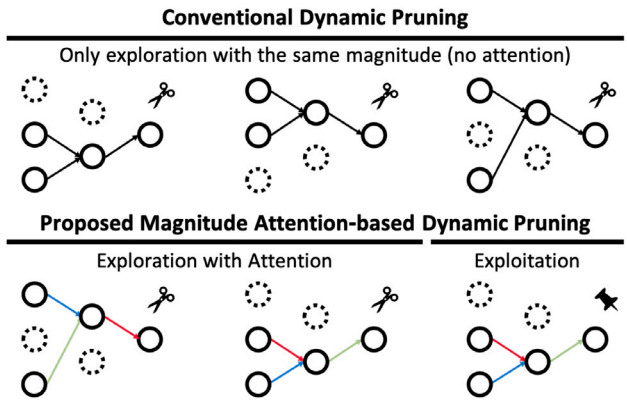

Fig. 2. MAP vs. conventional dynamic pruning in a training timeline perspective.

magnitude weights until the desired pruning ratio is met. The resulting pruning mask $M = [m_1, \dots, m_n] \subset \mathbb{R}^n$ is defined as $M \in \{0,1\}^n$, where each mask element m_j is determined based on whether the corresponding weight w_j exceeds the threshold set by the pruning ratio P.

Once the pruning mask is calculated, the sparse model $\overline{W}$ is constructed by element-wise multiplication of the mask with the model weights W, *i.e.*, $\overline{W} = M \odot W$. Given the pruning ratio P, the dataset D consisting of individual samples (x_k, y_k), and the sparse model parameterized by $M \odot W$, the optimization problem can be expressed as follows:

$$\min_{W} \frac{1}{B} \sum_{i}^{B} \mathcal{L}(f(M \odot W; x_k), y_k) \quad s.t. \quad \|W\|_0 \leq (1-P) \cdot n \tag{1}$$

where B is the batch size and n is the number of weights. $\|\cdot\|_0$ denotes the L_0 norm. Dynamic pruning (Guo et al., 2016; Lin et al., 2020) not only trains the sparse model but also jointly evolves the unimportant weights with the straight-through estimator (STE) (Bengio et al., 2013). The update rule for the jth weight with the objective function in Eq. (1) is as follows:

$$w_j^{i+1} = w_j^i - \eta \frac{\partial \mathcal{L}}{\partial \overline{w}_j^i} \frac{\partial \overline{w}_j^i}{\partial w_j^i} \approx w_j^i - \eta \frac{\partial \mathcal{L}}{\partial \overline{w}_j^i}, \ \forall j \in \{1, \dots, n\} \tag{2}$$

where i is the iteration. When the pruning mask of the jth weight is zero, *i.e.*, $\partial \overline{w}_j^i / \partial w_j^i = m_j = 0$, the unimportant weight (w_j^i) cannot be updated. However, dynamic model pruning methods approximate $\partial \overline{w}_j^i / \partial w_j^i = 1$ to update unimportant weights for dynamically searching the structure of the sparse model.

Gradual Pruning. In contrast to one-shot pruning, which prunes the model to achieve a target pruning ratio in a single iteration and can result in performance degradation without retraining, gradual pruning incrementally increases the current pruning ratio based on the current epoch (Zhu & Gupta, 2017). Gradual pruning is widely employed in sparsity-during-training since it better discovers a dynamic sparse structure than a one-shot approach. In this approach, the current pruning ratio (P_c) is gradually increased from an initial pruning ratio (P_s at $c = c_0$) to a target pruning ratio (P_t) as the current epoch (c) progresses.

$$P_c = P_t + (P_s - P_t)(1 - \frac{c - c_0}{E})^3, \quad c \in \{c_0, \dots, c_0 + E\}. \tag{3}$$

where E is the total number of epochs. We adopt this gradual pruning schedule in our method.

Pruning Threshold Computation. The pruning threshold in our method is determined by the magnitudes of the model weights, following the widely used approach introduced in Han et al. (2015). Specifically, the weights are sorted by magnitude, and the threshold is set to prune the smallest magnitude weights until the desired pruning ratio P_c is reached. This threshold dynamically changes during the training process as P_c increases, aligning with the gradual pruning schedule described in Eq. (3). The pruning mask is then computed as:

$$m_j = \begin{cases} 1, & \text{if } |w_j| > T(P_c) \\ 0, & \text{otherwise,} \end{cases} \tag{4}$$

where $T(P_c)$ denotes the dynamic threshold for the current pruning ratio P_c. By dynamically adjusting the threshold, our method ensures that pruning decisions are made progressively, enabling the discovery of an optimal sparse structure without sacrificing model performance.

This approach aligns with the gradual pruning strategy of DPF, which also utilizes weight magnitude for pruning. To avoid ambiguity, we have explicitly clarified this in the revised manuscript to ensure the pruning threshold computation is transparent and consistent.

4. Method

Magnitude Attention-based Dynamic Model Pruning As we adopt both dynamic and gradual model pruning, the pruning ratio changes in each epoch. Following Lin et al. (2020), we also modify the mask $M = [m_1, \dots, m_n] \subset \mathbb{R}^n$ for each $\mathcal{F}$ iteration within the same epoch to search the sparse structure. Consequently, we can define the pruning mask based on the current iteration (i) and epoch (c). We note that the pruning ratio depends on the current epoch and the pruning mask is updated every F iteration based on the threshold at ith iteration. The pruning threshold $\lambda^{(i,P_c)}$ is computed according to the current pruning rate based on the epoch (Eq. (3)). The pruning mask of jth weight at the ith iteration, $m_j^{(i,P_c)}$, can be calculated as follows:

$$\begin{aligned} m_j^{(i,P_c)} &= \begin{cases} Mask(\lambda^{(i,P_c)}, w_j^i) & \text{if} \quad i \,\%\, \mathcal{F} == 0 \\ m_j^{(i-1,P_c)} & \text{otherwise,} \end{cases} \\ Mask(\lambda^{(i,P_c)}, w_j^i) &= \begin{cases} 1 & \text{if} \quad |w_j^i| > \lambda^{(i,P_c)} \\ 0 & \text{otherwise,} \end{cases} \end{aligned} \tag{5}$$

The proposed MAP consists of a magnitude attention-based exploration and exploitation upon an explored sparse structure. We explain two parts in the following sections.

Exploration with a Magnitude Attention. In this work, we develop our magnitude attention approach based on three principles. **(1)** During the early stages of learning, due to a low pruning ratio, we facilitate effective exploration by updating both important and unimportant weights with similar magnitudes. As the pruning ratio increases, we shift the focus towards updating important weights rather than unimportant ones. **(2)** We distinguish and update important weights based on their magnitudes in the forward and backward paths. **(3)** To further enhance exploration, we treat the attention values of unimportant weights uniformly and assign them to be contiguous with those of important weights.

To adhere to these three principles, in forward path, we multiply important weights by normalized values, ranging in $[(1-P_c)^z, 1]$, while

Algorithm 1 Magnitude Attention-based Dynamic Pruning (MAP)

Require: Mask update frequency: $\mathcal{F}$, Pruning mask: $M \in \{0,1\}^n$, Model weights: $W \in \mathbb{R}^n$, Total iteration: T. Target exploitation iteration: K ▷ n is the number of parameters

for $i = 1$ to T **do** ▷ i is the index of iteration
  if i mod $\mathcal{F} == 0$ **and** $i \leqslant K$ **then** ▷ After K iteration, it enters the exploitation phase.
    Update pruning mask M^i with Eq. (5)
    Update magnitude attention A^i with Eq. (6)
  end if
  Compute a sparse model $\overline{W}^i = M^i \odot W^i$ ▷ $\odot$ represents Hardmard product
  Apply magnitude attention to the sparse model $\widetilde{W}^i = A^i \odot \overline{W}^i$ ▷ In exploitation phase, M^i and A^i are fixed
  Update weights $W^{i+1} = W^i - \eta \frac{\partial \mathcal{L}}{\partial \widetilde{W}^i} \odot A^i$
end for

unimportant weights are removed by a zero-valued mask. In backward path, at the beginning of training when the pruning ratio is low, both important and unimportant weights are updated similarly. As learning progresses, we concentrate on the identified (sparse) structure by decreasing the update degree of unimportant weights, which helps to prioritize and refine the most relevant connections, leading to improved performance.

At every ith iteration, the magnitude attention value $A^i = [a_1^i, \dots, a_n^i]$ is applied to the model weights. The value a_j^i, which corresponds to the jth weight w_j^i, is defined as follows:

$$a_j^i = \begin{cases} Att(w_j^i, m_j^{(i,P_c)}) & \text{if} \quad i \,\%\, \mathcal{F} == 0 \\ a_j^{i-1} & \text{elsewise,} \end{cases}$$
$$Att(w_j^i, m_j^{(i,P_c)}) = \begin{cases} b_j^i & \text{if} \quad m_j^{(i,P_c)} == 1 \\ (1-P_c)^z & \text{elsewise,} \end{cases} \tag{6}$$

where,

$$b_j^i = \frac{|w_j^i| - min(|W^i|)}{max(|W^i|) - min(|W^i|)} \cdot (1-(1-P_c)^z) + (1-P_c)^z$$

where $max(|W^i|)$ and $min(|W^i|)$ are the element-wise absolute max and min values of the model weights at ith iteration (W^i). z is an attention strength value that controls the range of attention. We will discuss the effect of z in Section 5.3.4. As depicted in Fig. 1, magnitude attention of important weights are in the range of $[(1-P_c)^z, 1]$ and unimportant weights are calculated by $(1-P_c)^z$.

In forward propagation, we multiply magnitude attention to the sparse model (*i.e.* $\overline{w}_j^i = m_j^{(i,P_c)} \cdot w_j^i$ and $\widetilde{w}_j^i = a_j^i \cdot \overline{w}_j^i$) and calculate the loss function to update model weight. As a result, the update rule of the proposed magnitude attention-based dynamic model pruning can be written as follows:

$$w_j^{i+1} = w_j^i - \eta \frac{\partial \mathcal{L}}{\partial \widetilde{w}_j^i} \frac{\partial \widetilde{w}_j^i}{\partial \overline{w}_j^i} \frac{\partial \overline{w}_j^i}{\partial w_j^i} \approx w_j^i - \eta \frac{\partial \mathcal{L}}{\partial \widetilde{w}_j^i} \cdot a_j^i, \ \forall j \in \{1, \dots, n\} \tag{7}$$

In backward propagation, as depicted in Eq. (7), the attention value is multiplied by gradients. Fig. 1 shows an overview of the exploration phase with magnitude attention considering the importance of weights in forward and backward paths. This double-sided attention mechanism places a higher priority on updating layers that have a substantial effect on the model's performance. By concentrating on the critical layers and providing them with targeted updates, we can enhance the overall performance of the model.

Exploitation with an Explored Sparse Structure. Typical dynamic pruning methods employ STE (Bengio et al., 2013) to uncover sparse structures throughout the entire training process. While this approach is more effective than methods that disregard the updating of unimportant weights, it suffers from learning instability due to the gradient approximation effect of STE.

To address this, our method employs a two-phase approach: during the exploration phase, the pruning mask and attention values are dynamically updated to iteratively refine the sparse structure. In the exploitation phase, the sparse structure is fixed to stabilize learning, while the backward pass continues to update all weights, including both important and pruned weights. This ensures that the model retains dynamic characteristics throughout training, as all weights contribute to the optimization process. We conjecture that fixing the sparse structure in the exploitation phase, while dynamically updating all weights, further elaborates on *exploitation* and enables well-explored sparse models to achieve improvements in both performance and robustness.

Fig. 2 compares conventional dynamic model pruning to ours in terms of the entire training timeline. MAP first explores the sparse model with a novel magnitude attention module. After K epochs, the attention values and sparse masks are fixed, and the model enters the exploitation phase, where the fixed sparse structure is trained dynamically by updating all weights. Algorithm 1 outlines the overall process of the proposed MAP method.

5. Experiments

5.1. Experimental setup

Datasets & Backbones. We evaluate the performance of pruning methods on three datasets: CIFAR-10/100 (Krizhevsky, Hinton et al., 2009) and ImageNet (Deng et al., 2009). All the methods prune ResNet (He, Zhang, Ren, & Sun, 2016), specifically utilizing 20, 32, and 56-layers of ResNet for CIFAR-10/100 while employing the 50-layer version for ImageNet assessments. For the CIFAR-10/100 benchmarks, which include comparison and model analysis, we present the average accuracy obtained from three runs.

Baselines. Our model is compared against various state-of-the-art pruning methods, including SM (Dettmers & Zettlemoyer, 2019), SNIP (Lee et al., 2018), DSR (Mostafa & Wang, 2019), SSC (Verma, Mehta, Si, Henao, & Carin, 2022), DPF (Lin et al., 2020), CP (Srinivas et al., 2022), GMP (Gupta et al., 2022), BIMP (Zimmer, Spiegel, & Pokutta, 2023), STR (Kusupati et al., 2020), and DST (Liu, Xu, Shi, Cheung, & So, 2020).

Implementation details. To conduct a rigorous comparison, we meticulously followed the setups from DPF (Lin et al., 2020) for CIFAR-10/100 and STR (Kusupati et al., 2020) for ImageNet. Our implementation incorporates magnitude-based unstructured weight pruning (Han et al., 2015) throughout all layers of the neural network, consciously avoiding layer-specific pruning. We apply the proposed pruning method and others to the entire network except for the batch normalization (Ioffe & Szegedy, 2015) and final fully connected layers.

For the CIFAR-10/100 datasets (Krizhevsky et al., 2009), we train all the models for 300 epochs, with an initial learning rate of 0.2 and an L2 weight decay of $1e-4$. The learning rate is decreased by a factor of 10 at 150 and 225 epochs, with the latter being the designated epoch for pruning. In case we use the exploitation phase (Section 4), we freeze the sparse structure after 250 epochs. We use a batch size of 128 (with a single P40 GPU) and set the mask update frequency (F) to 16, following DPF (Lin et al., 2020).

With regard to ImageNet, we train the model for 100 epochs, incorporating a 10-epoch warm-up phase with an initial learning rate of 0.256. The learning rate is controlled by a cosine LR schedule

Table 1
Comparison of pruning methods on CIFAR-10.
Here, we report Top-1 test accuracy by selecting the highest score achieved for each method. Underlined values indicate instances where the pruned model outperforms the dense one. N/A denotes the method cannot converge.

Backbone			**ResNet-20**	
Sparsity	70%	80%	90%	95%
Dense		92.36± 0.10		
SM (Dettmers & Zettlemoyer, 2019)	91.98± 0.01	91.54± 0.16	89.76± 0.40	83.03± 0.74
DSR (Mostafa & Wang, 2019)	92.00± 0.19	91.78± 0.28	87.88± 0.04	N/A
DPF (Lin et al., 2020)	92.42± 0.14	92.17± 0.21	90.88± 0.07	88.01± 0.30
CP (Srinivas et al., 2022)	-	-	90.90± 0.10	89.29± 0.20
MAP (ours)	**92.72**± 0.20	**92.30**± 0.13	**92.19**± 0.04	**90.64**± 0.32
Backbone			**ResNet-32**	
Sparsity	70%	80%	90%	95%
Dense		93.22± 0.07		
SNIP (Lee et al., 2018)	–	–	90.40± 0.26	87.23± 0.29
SM (Dettmers & Zettlemoyer, 2019)	–	–	91.54± 0.18	88.68± 0.22
DSR (Mostafa & Wang, 2019)	–	–	91.41± 0.23	84.12± 0.32
DPF (Lin et al., 2020)	–	–	92.42± 0.18	90.94± 0.35
GMP (Gupta et al., 2022)	–	–	92.67± 0.03	90.65± 0.13
MAP (ours)	93.32± 0.17	93.26± 0.23	**93.13**± 0.03	**92.16**± 0.11
Backbone			**ResNet-56**	
Sparsity	70%	80%	90%	95%
Dense		94.34± 0.19		
SM (Dettmers & Zettlemoyer, 2019)	–	–	92.73± 0.21	90.96± 0.40
DSR (Mostafa & Wang, 2019)	–	–	93.78± 0.20	92.57± 0.09
DPF (Lin et al., 2020)	–	–	93.95± 0.11	92.74± 0.08
BIMP (Zimmer et al., 2023)	–	–	93.35± 0.13	92.57± 0.32
MAP (ours)	94.35± 0.04	93.90± 0.16	**94.02**± 0.04	**93.17**± 0.13

Table 2
Comparison of pruning methods on CIFAR-10 with VGG-16 and CIFAR-100 with DenseNet-121.
Top-1 test accuracy for each method is reported, with the "Difference" column showing the accuracy change relative to the dense model.

Backbone	VGG-16 (CIFAR-10)		
Method	Dense	Accuracy	Difference
SNP (Lee et al., 2018)	93.74± 0.13	93.04± 0.26	−0.7
SM (Dettmers & Zettlemoyer, 2019)	93.74± 0.13	93.59± 0.17	−0.15
DPF (Lin et al., 2020)	93.74± 0.13	93.87± 0.15	+ 0.13
MAP (ours)	93.25± 0.29	93.86± 0.11	+ **0.61**
Backbone	**DenseNet-121 (CIFAR-100)**		
Method	Dense	Accuracy	Difference
DPF (Lin et al., 2020)	77.63± 0.44	77.14± 0.18	−0.49
MAP (ours)	77.63± 0.44	77.65± 0.18	+ **0.02**

Table 3
Comparison of pruning methods on CIFAR-100 (ResNet-32) in terms of Top-1 Accuracy.

Backbone	ResNet-32	
Sparsity	90%	95%
Dense	74.64	
DSR (Mostafa & Wang, 2019)	69.63	68.20
SET (Mocanu, Mocanu, Stone, Nguyen, Gibescu et al., 2018)	69.66	67.41
Deep-R (Bellec, Kappel, Maass, & Legenstein, 2018)	66,78	63.90
SNIP (Lee et al., 2018)	68.89	65.22
GraSP (Wang, Zhang, & Grosse, 2020)	69.24	66.50
Dense	70.60	
DPF (Lin et al., 2020)	67.15	62.97
MAP (ours)	**70.42**	**67.23**

The table presents results for ResNet-32 on the CIFAR-100 dataset.

(Loshchilov & Hutter, 2016). The mask is updated at every iteration ($F = 1$). We implement an L1 weight decay of $2e - 5$. All experiments are conducted using SGD with a Nesterov momentum of 0.875. The model is trained using four NVIDIA A100 GPUs, achieving a batch size of 1024. The model reaches the target sparsity at 60 epochs, and the sparse structure is fixed at 80 epochs.

Table 4
Comparison of pruning methods on ImageNet (ResNet-50) in terms of Top-1 Accuracy.

Backbone	ResNet-50	
Sparsity	80%	90%
Dense	75.95	
Incre. (Zhu & Gupta, 2017)	74.25	73.36
SM (Dettmers & Zettlemoyer, 2019)	74.59	72.65
DSR (Mostafa & Wang, 2019)	73.30	71.60
STR (Kusupati et al., 2020)	70.70	70.13
GSM (Ding et al., 2019)	72.75	70.08
LC (Carreira-Perpinan & Idelbayev, 2018)	73.87	67.57
DST (Liu et al., 2020)	73.16	71.35
GMP (Gupta et al., 2022)	74.19	72.80
BIMP (Zimmer et al., 2023)	75.08	73.53
DPF (Lin et al., 2020)	75.12	74.38
DNW (Wortsman, Farhadi, & Rastegari, 2019)	75.27	74.29
CP (Srinivas et al., 2022)	75.30	73.30
MAP (ours)	**75.90**	**74.90**

The table presents results for ResNet-50 on the ImageNet dataset.

In summary, our models are reached at target sparsity (Target pruning ratio reached epoch) at 225, 60 epoch and freeze the mask for exploitation (K epoch) at 250 epoch and 80 epoch on CIFAR and ImageNet dataset, respectively.

5.2. Quantitative comparison

Tables 1 and 2 presents the top-1 test accuracy of various pruning methods on CIFAR-10. Our proposed method, MAP, consistently outperforms its competitors, demonstrating the effectiveness of magnitude attention. For instance, compared to the second-best methods (*e.g.*, CP (Srinivas et al., 2022) and DPF (Lin et al., 2020)) at 95% sparsity, MAP shows a performance improvement of over 1% for both the 20 and 32-layer ResNet and a 0.4% increase in ResNet-56. Notably, at low sparsity levels (*e.g.*, 70%), MAP surpasses the performance of dense networks. As depicted in Table 3, MAP exceeds the other pruning methods on CIFAR-100 as well, similar to the CIFAR-10 benchmark. MAP also outperforms the dense network in low sparsity cases on

Table 5
Pruning configurations and their detailed descriptions in the internal study.
We manipulate a weight mask in the forward path and employ different weighting schemes for sparse and non-sparse models in the backward path. The mathematical representations of each pruning configuration are described, including a new configuration, Config D w/o FA, where the proposed attention is only applied in the backward path, excluding the forward attention present in Config D. (Note: $(1-P_c)^z$ is applied to non-sparse weights to gradually reduce their update magnitude, emphasizing the updates for sparse weights during training.)

Config	Forward path	Backward path
A	$\overline{w}^i_j = m^{(i,P_c)}_j \cdot w^i_j$ Sparse with a binary mask (Eq. (5)).	$w^{i+1}_j = w^i_j - \eta \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j} \frac{\partial \overline{w}^i_j}{\partial w^i_j}$ Sparse with a binary mask (Eq. (5)).
B	$\overline{w}^i_j = m^{(i,P_c)}_j \cdot w^i_j$ Sparse with a binary mask (Eq. (5)).	$w^{i+1}_j = w^i_j - \eta \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j}$ All layers (sparse and non-sparse) equally.
C	$\overline{w}^i_j = m^{(i,P_c)}_j \cdot w^i_j$ Sparse with a binary mask (Eq. (5)).	$w^{i+1}_j = \begin{cases} w^i_j - \eta \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j}, & \text{if } w^i_j \text{ is sparse} \\ w^i_j - \eta (1-p_c)^z \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j}, & \text{if } w^i_j \text{ is non-sparse} \end{cases}$ Sparse with full weight, non-sparse with $(1-p_c)^z$.
D w/o FA	$\overline{w}^i_j = m^{(i,P_c)}_j \cdot w^i_j$ Sparse with a binary mask (Eq. (5)).	$w^{i+1}_j = \begin{cases} w^i_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j} \cdot a^i_j, & \text{if } w^i_j \text{ is sparse} \\ w^i_j - \eta (1-p_c)^z \frac{\partial \mathcal{L}}{\partial \overline{w}^i_j}, & \text{if } w^i_j \text{ is non-sparse} \end{cases}$ Sparse with Eq. (6), non-sparse with $(1-p_c)^z$.
D	$\widetilde{w}^i_j = a^i_j \cdot \overline{w}^i_j$ Sparse with an attention mask (Eq. (6)).	$w^{i+1}_j = \begin{cases} w^i_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial \widetilde{w}^i_j} \cdot a^i_j, & \text{if } w^i_j \text{ is sparse} \\ w^i_j - \eta (1-p_c)^z \frac{\partial \mathcal{L}}{\partial \widetilde{w}^i_j}, & \text{if } w^i_j \text{ is non-sparse} \end{cases}$ Sparse with Eq. (6), non-sparse with $(1-p_c)^z$.

Table 6
Analysis of the effect of hyperparameter K and exploitation epochs on ResNet-20, ResNet-32, and ResNet-56 performance with the CIFAR-100 dataset.

Model	$K = 250$	$K = 260$	$K = 280$	$K = 290$
ResNet-20	68.10± 0.23	68.44± 0.10	**68.50**± 0.15	68.04± 0.08
ResNet-32	**70.42**± 0.16	70.40± 0.14	70.41± 0.15	70.13± 0.34
ResNet-56	71.90± 0.28	**72.24**± 0.13	72.01± 0.17	71.80± 0.20

CIFAR-100; our method achieves 70.93% accuracy at 80% sparsity, while the dense network shows 70.60%.

In Table 4, we compare various pruning methods on the ImageNet dataset at sparsity levels of 80% and 90%. Our proposed MAP consistently delivers outstanding accuracy, outperforming other methods by approximately 0.52% points (*e.g.*, 74.38% vs. 74.90%) in the 90% sparsity case. Furthermore, even at 80% sparsity, MAP's performance (75.90%) is nearly on par with the dense baseline (75.95%), underscoring the effectiveness of our proposed method.

5.3. Model analysis

In this section, we delve into the analysis of our proposed model, focusing on five key aspects that contribute to its overall performance. We begin by exploring the learning mechanism of MAP in Section 5.3.1, where we dissect the importance of mask in forward path and weight term in backward path to make an effective pruning method. In Section 5.3.2, we explore the effectiveness of the exploitation phase in our dynamic pruning strategy. We then compare various pruning configurations in Section 5.3.3 to understand the impact of different setups on the model's performance. In Section 5.3.4, we investigate the impact of attention strength on the model's ability to identify and enhance high-performing sparse networks. Finally, in Section 5.3.5, we examine the combined effects of using both attention and exploitation in our proposed methodology. Unless mentioned otherwise, our analysis is conducted using the ResNet-56 backbone on the CIFAR-100 dataset with a pruning ratio of 90%. We also broaden our investigation to various training setups, backbones, and datasets to validate the robustness and adaptability of our approach.

5.3.1. Understanding the learning mechanism of MAP

Table 5 presents four distinct configurations (Config A, B, C, and D) along with an additional setup (Config D w/o FA) to analyze the learning mechanism of MAP. For each setup, we differentiate between a mask for the sparse model, employed during the forward path, and a weighting scheme for both sparse and non-sparse models, utilized in the backward path. Config A incorporates a binary mask for both forward and backward paths (Zhu & Gupta, 2017). In this configuration, the binary mask is utilized to selectively prune weights in both directions. Config B is similar to Config A in the forward path as it employs a sparse binary mask. However, for the backward path, it adopts the straight-through estimator (STE) (Bengio et al., 2013) that enables equivalent updates of both sparse and non-sparse models, akin to the DPF mechanism (Lin et al., 2020). Config C also uses a binary mask in the forward path, however, it employs $(1-p_c)^z$ weighting scheme for the non-sparse model in the backward path. By doing so, it progressively decreases the concentration of the non-sparse model when the pruning ratio is gradually increased. Config D w/o FA (Forward Attention) is an additional setup that applies the proposed attention mechanism exclusively in the backward path while using a binary mask for the forward path. This configuration helps understand the importance of applying attention in both paths by comparing it with Config D. Finally, Config D is our proposed MAP method, which leverages a magnitude attention mask (Eq. (6)) for the sparse model in both forward and backward paths. This approach prioritizes critical layers and enhances the efficacy of sparse model updates. Additionally, the non-sparse model is updated using the $(1-p_c)^z$ weighting scheme, fostering proactive exploration during training.

Hyper parameter K and attention A^i. We conducted experiments to analyze the impact of the masking freeze starting epoch K on model performance. In the default experiments, K was set to 250, and additional tuning experiments were performed at intervals of 10 epochs, as shown in Table 6. While the performance differences were not significant, the analysis in Section 5.4.1 highlights the critical role of exploitation enabled by masking freeze.

After epoch K, A^i is no longer updated and remains fixed at the point where masking freeze is applied. This is because both A^i and the mask pattern are treated as part of the sparse structure, and they are updated together whenever the frequency changes. When the mask is modified, A^i is also updated accordingly.

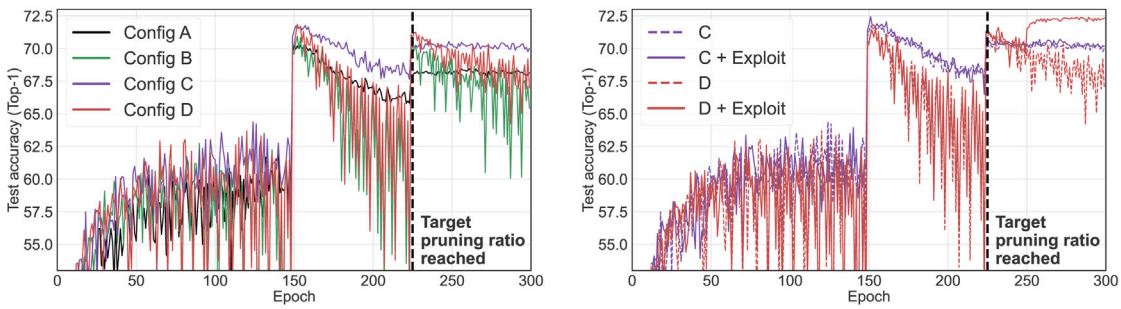

Fig. 3. (Left) Test accuracy during ResNet-56 training on CIFAR-100 for CONFIG A-D, showcasing the performance of different pruning configurations throughout the training process. (Right) Test accuracy during the training process of CONFIG C and D with exploitation, demonstrating the effectiveness of the pruning method when shifting from exploration to exploitation.

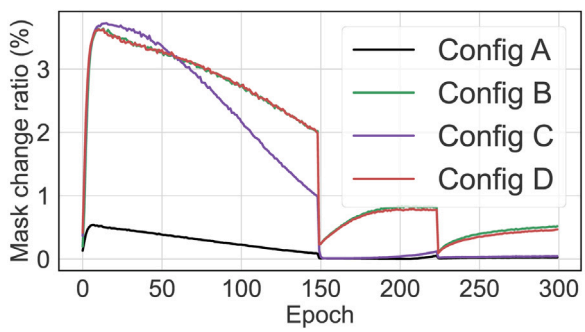

Fig. 4. Ratio of pruning mask changes. We visualize Config A-D (Section 5.3.1)

Table 7
Comparison of fixed and dynamic attention values during exploitation mode.

Model	ResNet-20	ResNet-32	ResNet-56
Fixed Attention	68.50± 0.15	**70.42**± 0.14	**72.24**± 0.13
Dynamic Attention	**68.54**± 0.01	70.38± 0.14	72.16± 0.23

Table 8
Comparison of Config D (Ours) and Config D (No Attention During Backward Phase) across ResNet models.

Configuration	ResNet-20	ResNet-32	ResNet-56
Config D (w/o Backward Attention)	67.64± 0.17	69.73± 0.20	71.18± 0.09
Config D	**68.50**± 0.15	**70.42**± 0.14	**72.24**± 0.13

We also experimented with a real-time updating approach for A^i, and the results, included in Table 7, show that synchronizing A^i updates with mask changes yields similar performance, indicating no significant advantage of real-time updates.

The effect of backward attention. The reason for applying A_i during the backward phase in Config D is that A_i is multiplied during the forwarding phase. Therefore, when differentiating with respect to W, A_i is naturally multiplied in the backward function as well. By utilizing STE, the dynamic effect can be achieved, which aligns with the current motivation behind the MAP method. Experimentally, there were no cases where convergence failed due to STE. Additionally, an experiment was conducted where attention was disabled during the backward phase (Config D without backward attention), and the results are included in Table 8. The results indicate that the Config D approach, where A_i is naturally multiplied during the backward phase due to its application in the forward phase, was more effective.

Weighting scheme for the non-sparse model. Although dynamic pruning (*e.g.* DPF (Lin et al., 2020)) demonstrates its strengths in the exploration of sparse models, it often updates non-sparse models more frequently than necessary due to the equal weight update given to both sparse and non-sparse candidates. To investigate this, we compare Config B, which has the same setting as DPF and Config C, which downweights the non-sparse model during the backward path using $(1-p_c)^z$ weighting scheme. As shown in Fig. 3 (left), the test accuracy in the early stages of training (< 150 epoch) are similar for both configurations as they explore the sparse candidates in the same manner. However, a noticeable difference in the accuracy emerges in the mid-to-late stages of training (> 150 epoch). Note that MAP utilizes both exploration and exploitation simultaneously. It builds upon the Config D method by incorporating exploitation starting from the K epoch. This corresponds to the "Config D + exploitation" approach shown on the right side of Fig. 3.

We conjecture that this occurs because we guide Config C to update sparse model more frequently at the later training phase, leading to stabilized learning. Fig. 4 visualizes the extent of changes in the sparse candidates during the training process using the sparse mask change ratio. While Config B consistently exhibits a high change ratio, Config C initially shows a high ratio that rapidly declines as training progresses. Consequently, we can infer that Config C modifies the weighting scheme for non-sparse models in the backward path, thereby reducing the exploration effect. Such alteration naturally strengthens the exploitation of sparse models (as we consider the exploration–exploitation trade-off), resulting in an overall improvement in model performance and robustness than Config B (and DPF (Lin et al., 2020)).

Magnitude attention. The weighting scheme for non-sparse models (Config C) exhibits higher performance and stability compared to Config B. Nevertheless, it has the drawback of significantly reducing the exploration effect in the later stages of training (Fig. 4). Reducing the exploration power may limit the interchange between sparse and non-sparse models, which potentially poses the risk of getting stuck in local optima. In addition, our proposed exploitation method (Section 4) requires substantial exploration to achieve optimal results. To investigate an effective way of improving exploration without sacrificing performance, we compare Config C to Config D, which employs the proposed magnitude attention in both the forward and backward paths.

Table 9
Comparison of pruning configurations on CIFAR-10.
We report test accuracy using ResNet-20/32/56 (shortly R20/R32/R56) backbones. All the results include the last and best epochs' scores to assess the training stability (smaller deviation indicates higher stability).

		Dense	Exploit?	A	B	C	D w/o FA	D
R20	Last	92.22± 0.15	✗	90.79± 0.07	87.84± 1.91	**90.90**± 0.05	88.07± 0.44	88.32± 1.19
			✓	90.81± 0.01	91.35± 0.17	91.02± 0.16	91.51± 0.11	**92.12**± 0.06
	Best	92.36± 0.10	✗	91.03± 0.06	90.79± 0.07	**91.13**± 0.12	91.03± 0.02	91.36± 0.12
			✓	91.10± 0.03	91.47± 0.22	91.35± 0.20	91.64± 0.13	**92.19**± 0.04
R32	Last	93.16± 0.01	✗	91.93± 0.21	91.01± 0.33	**92.24**± 0.15	90.77± 0.27	91.79± 0.24
			✓	92.02± 0.20	92.42± 0.29	92.05± 0.27	92.79± 0.13	**92.97**± 0.11
	Best	93.22± 0.07	✗	92.14± 0.13	92.30± 0.16	92.50± 0.16	92.54± 0.10	**92.73**± 0.01
			✓	92.19± 0.05	92.56± 0.28	92.33± 0.35	92.86± 0.16	**93.18**± 0.03
R56	Last	93.86± 0.11	✗	92.82± 0.37	92.62± 0.12	**93.29**± 0.19	92.78± 0.03	93.18± 0.12
			✓	93.19± 0.17	93.31± 0.35	93.31± 0.08	93.52± 0.02	**93.84**± 0.08
	Best	94.03± 0.16	✗	93.00± 0.53	93.16± 0.09	93.51± 0.23	93.18± 0.13	**93.54**± 0.07
			✓	93.34± 0.20	93.44± 0.30	93.59± 0.05	93.61± 0.04	**94.02**± 0.04

Table 10
Comparison of pruning configurations on CIFAR-100.
We report test accuracy using ResNet-20/32/56 (shortly R20/R32/R56) backbones. All the results include the last and best epochs' scores to assess the training stability (smaller deviation indicates higher stability).

		Dense	Exploit?	A	B	C	D w/o FA	D
R20	Last	67.86± 0.41	✗	**65.21**± 0.36	52.55± 2.58	64.99± 0.36	54.01± 0.65	57.62± 1.42
			✓	65.61± 0.13	65.63± 0.27	66.13± 0.40	65.89± 0.25	**67.95**± 0.34
	Best	68.64± 0.22	✗	65.67± 0.36	63.79± 0.20	**65.86**± 0.28	63.95± 0.30	65.13± 0.51
			✓	65.99± 0.11	65.79± 0.26	66.46± 0.27	65.92± 0.32	**68.50**± 0.15
R32	Last	69.89± 0.21	✗	67.34± 0.38	56.87± 1.83	**67.94**± 0.27	57.45± 2.80	60.24± 2.58
			✓	67.45± 0.19	69.72± 0.18	68.44± 0.09	69.74± 0.14	**70.17**± 0.12
	Best	70.60± 0.07	✗	67.91± 0.42	67.88± 0.48	**68.75**± 0.23	67.70± 0.34	68.58± 0.52
			✓	68.00± 0.16	69.69± 0.10	68.79± 0.05	69.95± 0.19	**70.42**± 0.14
R56	Last	71.71± 0.64	✗	68.84± 0.72	65.32± 0.38	**69.77**± 0.20	64.29± 1.33	65.92± 1.28
			✓	68.86± 0.90	71.58± 0.14	70.17± 0.10	71.62± 0.05	**72.02**± 0.31
	Best	72.01± 0.31	✗	69.33± 0.84	70.44± 0.15	70.60± 0.13	70.89± 0.34	**70.93**± 0.31
			✓	69.43± 0.84	71.64± 0.11	70.79± 0.06	71.78± 0.04	**72.24**± 0.13

Table 11
Training of Config A-D with exploitation.
The difference in accuracy between the w/ and w/o exploitation.

Δ Accuracy w/o *Exploit*	A	B	C	D
Last	+ 0.02	+ 6.26	+ 0.40	+ 6.10
Best	+ 0.11	+ 1.21	+ 0.19	+ 1.32

The magnitude attention allows a more active transition between candidates by making the update rule for sparse and non-sparse models a unified continuous form. Additionally, this approach sorts connections by importance based on the model weight's magnitude, allocating more attention to critical models. As shown in Fig. 4, Config D successfully raises the low exploration of Config C to that of the original dynamic pruning method (Config B; DPF) while maintaining overall learning performance and stability (Fig. 3 (left)).

In essence, compared to the static or dynamic pruning baseline (Config A or B), our method (Config D), which incorporates magnitude attention and a weighting scheme for non-sparse models, achieves significant improvements in learning performance and stability while maintaining an equivalent exploration perspective. This advancement can be considered a substantial development, taking into account the exploration–exploitation trade-off.

5.3.2. Understanding the exploitation on dynamic pruning

In Section 4, we improved the exploitation stage in the dynamic pruning approach. This method involves freezing the pruning mask in the later stages of training to prevent any changes in the sparse network, allowing only the sparse model to be updated while the final network structure remains fixed. By maximizing exploitation in this manner, the overall performance and robustness of the learning process are significantly increased.

We note that by their design, Config A, C focus less on exploration, while Config B, D enhance exploration. For example, Config C minimizes exploration to prioritize exploitation, while Config D significantly incorporates both exploration and exploitation. Although further exploitation is introduced, *less-explored* models (Config A, C) only show marginal performance improvement. We argue that this is because their restricted exploration makes additional exploitation ineffective and even results in overfitting.

In contrast, *well-explored* models (Config B, D) achieve a substantial performance improvement for both the final accuracy and the best accuracy achieved during training. The training curve (Fig. 3 (right)) also highlights the effectiveness of exploitation on the model performance; after the target pruning ratio is reached (which occurs at 225 epochs in this experiment), the well-explored Config D outperforms Config C when boosted by additional exploitation. Fig. 5 also illustrates the impact of exploitation by presenting the final accuracy at the end of the pruning process (Kim, Yoo, Song, Yoo, & Kwak, 2021). In these results, the models with extensive exploration (Config B, D) exhibit lower final accuracy (and higher deviation) before exploitation; however, their performance (and instability) are greatly improved when further exploitation is introduced. On the other hand, the models with insufficient exploration (Config A, C) attain only marginal improvement, even though they display higher final accuracy before exploitation. Our experiments reveal that exploitation is essential in the dynamic pruning methodology, and they showcase that our approach is optimally designed to balance exploration and exploitation, thereby enhancing performance and robustness (see Table 11).

Table 12
Test accuracy on various attention strengths z.
We present the accuracy on CIFAR-10 with ResNet-20/32 (R20/R32) backbones. The result demonstrates that higher attention strength z leads to improved performance of the pruned models.

$z =$	0.4	0.6	0.8	1.0	1.2	1.4	1.6
R20	88.20± 0.09	88.72± 0.06	89.03± 0.50	89.30± 0.41	90.00± 0.11	90.27± 0.12	90.60± 0.23
R32	91.12± 0.28	91.29± 0.14	91.82± 0.13	91.80± 0.13	92.11± 0.10	91.93± 0.48	92.47± 0.28

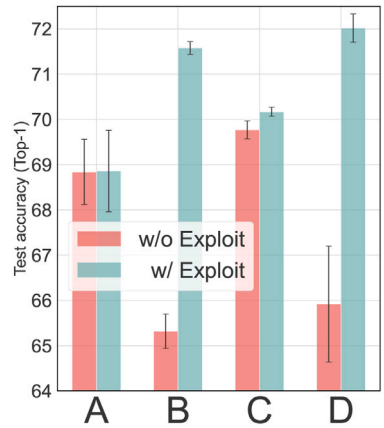

Fig. 5. Effect of exploitation in dynamic model pruning.

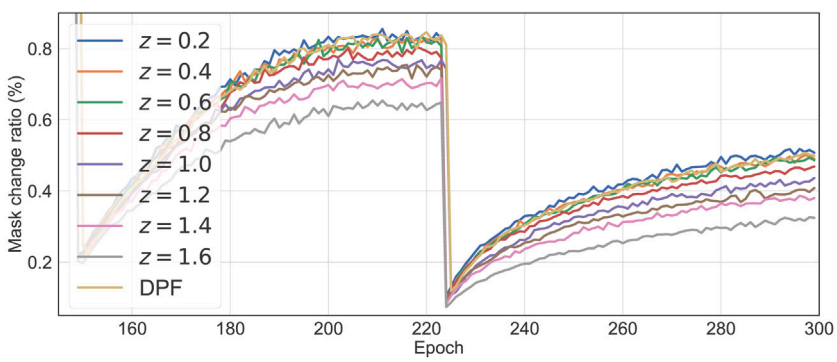

Fig. 6. Pruning mask change ratio on various attention strengths z.

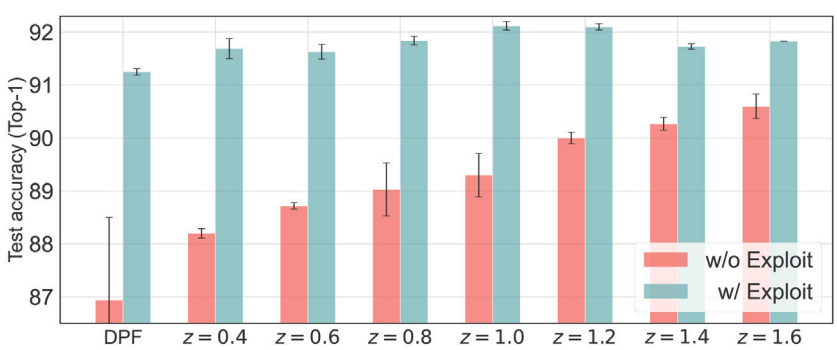

Fig. 7. Effect of attention strength w/ and w/o exploitation. We show the result on CIFAR-10 using ResNet-20 model. All the scores are obtained in the last epoch, following Kim et al. (2021).

5.3.3. Comparison of pruning configurations

Here, we report the full comparison results of each pruning configuration as depicted in Table 5. In this evaluation, we use ResNet-20/32/56 (He et al., 2016) (shortly R20/R32/R56) as the backbone network and we validate all the methods on the CIFAR-10/100 (Krizhevsky et al., 2009) datasets. We experiment with five different setups (A, B, C, D, and D w/o FA) recording both the last and best scores for each.

As shown in Tables 9 and 10, Config A and C primarily focus on training the sparse model (exploitation), resulting in stable learning and a small accuracy difference between the last and best scores (Kim et al., 2021). However, due to limited exploration, the further exploitation effect is not prominent in these configurations.

On the other hand, Config B, D, and D w/o FA effectively train both the non-sparse and sparse models, facilitating enhanced exploration. Consequently, when reaching the exploitation phase of fixing the struc-

ture of the sparse network in the later stages of training, a significant performance improvement is demonstrated. Nevertheless, we can also observe the huge deviation between the last and best accuracy due to the training instability derived by the higher exploration.

The proposed pruning method, Config D (MAP) outperforms previous dynamic pruning models such as DPF (Lin et al., 2020) (*i.e.* Config B) in terms of performance. This could be attributed to the magnitude attention mechanism's ability to update critical layers more efficiently. While the pruning method focuses on training important layers rather than sub-layers (similar to Config C), our approach allows for continuous updates to both sparse and non-sparse networks instead of discrete updates. This continuous updating facilitates enhanced exploration, resulting in remarkable performance and effective exploitation. Consequently, magnitude attention plays a crucial role in enabling effective learning, surpassing other models in terms of training efficacy and overall performance.

To further investigate the effect of the magnitude attention, we compare Config D and D w/o FA, which applies the attention mechanism only backward propagation. The comparison demonstrates the efficacy of introducing attention to both training paths. By incorporating attention in the backward path, we promote more frequent updates to important layers' weights. Utilizing attention in the forward path during the training process directs the model updates towards the guidance of significant layers, leading to performance improvement.

5.3.4. Impact of attention strength

The attention strength in our pruning method is controlled by the parameter z in the formulation $(1-p)^z$ of Eq. (6). A smaller z value results in a higher value of $(1-p)^z$, leading to relatively weaker attention. On the other hand, a larger z value decreases the value of $(1-p)^z$, resulting in a stronger and more focused attention effect (Table 12). Through experiments with various z values, we consistently observed that as z increases, the attention becomes more sensitive to the importance of weights, enabling a better focus on preserving and updating crucial connections. Furthermore, we found that higher z values corresponded to improved performance outcomes (Fig. 7). These results strongly suggest that our attention-based method excels at identifying high-performing sparse networks and effectively enhancing their performance, outperforming conventional dynamic pruning approaches.

5.3.5. Impact of using both attention and exploitation

To investigate the advantages of the proposed attention-based pruning with the combined effects of exploitation, we conduct a series of experimental analyses. The proposed attention methodology achieves effective training of sparse networks by employing differential weight updates based on the magnitude importance of weights. Moreover, continuous updates are ensured for both sparse and non-sparse networks, leading to improved exploration performance (Section 5.3.1). By virtue of our methodology designed to facilitate robust exploration, the structural integrity of sparse networks is preserved, leading to significant performance gains when exploitation is employed (Section 5.3.2).

Nevertheless, finding a sweet spot between exploration and exploitation could be challenging, owing to their inherent trade-off relationship. Remarkably, we observed that larger z values reduce the transition from non-sparse to sparse networks (Fig. 6). In other words, strengthening the attention effect can be seen as prioritizing exploitation rather than exploring the overall model, which may diminish the effectiveness of fixing the structure of a well-explored sparse network during the latter stages of training. Consequently, when both attention and exploitation are utilized, irrespective of attention strength, similar performance outcomes are observed (Fig. 7). Therefore, all experiments were conducted with a fixed z value of 1.0 for simplicity.

6. Conclusion

In this work, we have proposed a Magnitude Attention-based Dynamic Model Pruning (MAP) method, which considers element-wise magnitudes in computational paths, unlike conventional pruning methods that treat weights equally in the forward and backward paths. MAP consists of exploration and exploitation phases, where the exploitation phase has shown to be highly effective for dynamic pruning methods that undergo exploration. Our experiments primarily focus on image classification datasets, with no exploration in tasks NLP or real-world scenarios. Despite this, our insights into pruning's attention aspect and the balance between exploration and exploitation should benefit future research.

CRediT authorship contribution statement

Jihye Back: Draft writing, ImageNet experiments. **Namhyuk Ahn:** Writing related work, CIFAR-10 experiments. **Jangho Kim:** Overall experimental design, paper revision, ablation study.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was carried out with the support of "Cooperative Research Program for Agriculture Science and Technology Development (Project No. RS-2024-00332198)". Rural Development Administration, Republic of Korea and was supported by INHA UNIVERSITY Research Grant.

Data availability

We used open benchmark dataset.

References

Alvarez, J. M., & Salzmann, M. (2016). Learning the number of neurons in deep networks. *Advances in Neural Information Processing Systems, 29*.

Bellec, G., Kappel, D., Maass, W., & Legenstein, R. (2018). Deep rewiring: Training very sparse deep networks. In *6th international conference on learning representations, ICLR 2018, Vancouver, BC, Canada, April 30 – May 3, 2018, conference track proceedings*. OpenReview.net, URL: https://openreview.net/forum?id=BJ_wN01C-.

Bengio, Y., Léonard, N., & Courville, A. (2013). Estimating or propagating gradients through stochastic neurons for conditional computation. arXiv preprint arXiv:1308.3432.

Carreira-Perpinán, M. A., & Idelbayev, Y. (2018). "Learning-compression" algorithms for neural net pruning. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 8532–8541).

Carreira-Perpinan, M. A., & Idelbayev, Y. (2018). "Learning-compression" algorithms for neural net pruning. In *2018 IEEE/CVF conference on computer vision and pattern recognition* (pp. 8532–8541). http://dx.doi.org/10.1109/CVPR.2018.00890.

Chen, Y., Ma, Z., Fang, W., Zheng, X., Yu, Z., & Tian, Y. (2023). A unified framework for soft threshold pruning. In *International conference on learning representations*.

Chen, Y., Yu, Z., Fang, W., Ma, Z., Huang, T., & Tian, Y. (2022). State transition of dendritic spines improves learning of sparse spiking neural networks. In *International conference on machine learning* (pp. 3701–3715). PMLR.

Deng, J., Dong, W., Socher, R., Li, L. J., Li, K., & Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition* (pp. 248–255). IEEE.

Dettmers, T., & Zettlemoyer, L. (2019). Sparse networks from scratch: Faster training without losing performance. arXiv preprint arXiv:1907.04840.

Ding, X., Ding, G., Zhou, X., Guo, Y., Han, J., & Liu, J. (2019). Global sparse momentum SGD for pruning very deep neural networks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems: Vol. 32*, Curran Associates, Inc., URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/f34185c4ca5d58e781d4f14173d41e5d-Paper.pdf.

Frankle, J., & Carbin, M. (2018). The lottery ticket hypothesis: Finding sparse, trainable neural networks. arXiv preprint arXiv:1803.03635.

Guo, Y., Yao, A., & Chen, Y. (2016). Dynamic network surgery for efficient DNNS. *Advances in Neural Information Processing Systems*, *29*.

Gupta, M., Camci, E., Keneta, V. R., Vaidyanathan, A., Kanodia, R., Foo, C. S., Min, W., & Jie, L. (2022). Is complexity required for neural network pruning? A case study on global magnitude pruning. arXiv preprint arXiv:2209.14624.

Han, S., Pool, J., Tran, J., & Dally, W. (2015). Learning both weights and connections for efficient neural network. *Advances in Neural Information Processing Systems*, *28*.

Hayou, S., Ton, J. F., Doucet, A., & Teh, Y. W. (2021). Robust pruning at initialization. In *International conference on learning representations*.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning* (pp. 448–456). PMLR.

Kim, J. (2023). Quantization robust pruning with knowledge distillation. *IEEE Access*, *11*, 26419–26426.

Kim, J., Yoo, K., & Kwak, N. (2020). Position-based scaled gradient for model quantization and pruning. *Advances in Neural Information Processing Systems*, *33*, 20415–20426.

Kim, J., Yoo, J., Song, Y., Yoo, K., & Kwak, N. (2021). Dynamic collective intelligence learning: Finding efficient sparse model via refined gradients for pruned weights. arXiv preprint arXiv:2109.04660.

Krizhevsky, A., & Hinton, G. (2009). Learning multiple layers of features from tiny images. Toronto, ON, Canada.

Kusupati, A., Ramanujan, V., Somani, R., Wortsman, M., Jain, P., Kakade, S., & Farhadi, A. (2020). Soft threshold weight reparameterization for learnable sparsity. In *International conference on machine learning* (pp. 5544–5555). PMLR.

LeCun, Y., Denker, J., & Solla, S. (1989). Optimal brain damage. *Advances in Neural Information Processing Systems*, *2*.

Lee, N., Ajanthan, T., & Torr, P. H. (2018). Snip: Single-shot network pruning based on connection sensitivity. arXiv preprint arXiv:1810.02340.

Lin, T., Stich, S. U., Barba, L., Dmitriev, D., & Jaggi, M. (2020). Dynamic model pruning with feedback. In *International conference on learning representations*.

Liu, J., Xu, Z., Shi, R., Cheung, R., & So, H. K. H. (2020). Dynamic sparse training: Find efficient sparse network from scratch with trainable masked layers. In *International conference on learning representations*.

Loshchilov, I., & Hutter, F. (2016). SGDR: Stochastic gradient descent with warm restarts. ArXiv Preprint arXiv: arXiv:1608.03983.

Mocanu, D. C., Mocanu, E., Stone, P., Nguyen, P. H., Gibescu, M., & Liotta, A. (2018). Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science. *Nature Communications*, http://dx.doi.org/10.1038/s41467-018-04316-3.

Mostafa, H., & Wang, X. (2019). Parameter efficient training of deep convolutional neural networks by dynamic sparse reparameterization. In *International conference on machine learning* (pp. 4646–4655). PMLR.

Park, S., Lee, J., Mo, S., & Shin, J. (2020). Lookahead: A far-sighted alternative of magnitude-based pruning. arXiv preprint arXiv:2002.04809.

Singh, S. P., & Alistarh, D. (2020). Woodfisher: Efficient second-order approximation for neural network compression. *Advances in Neural Information Processing Systems*, *33*, 18098–18109.

Srinivas, S., Kuzmin, A., Nagel, M., van Baalen, M., Skliar, A., & Blankevoort, T. (2022). Cyclical pruning for sparse neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 2762–2771).

Verma, V., Mehta, N., Si, S., Henao, R., & Carin, L. (2022). Pushing the efficiency limit using structured sparse convolutions. In *IEEE workshop/winter conference on applications of computer vision*. http://dx.doi.org/10.1109/WACV56688.2023.00644.

Wang, C., Zhang, G., & Grosse, R. (2020). Picking winning tickets before training by preserving gradient flow. In *International conference on learning representations*. URL: https://openreview.net/forum?id=SkgsACVKPH.

Wortsman, M., Farhadi, A., & Rastegari, M. (2019). Discovering neural wirings. *Advances in Neural Information Processing Systems*, *32*.

Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, *64*(3), 107–115.

Zhang, D., Wang, H., Figueiredo, M., & Balzano, L. (2018). Learning to share: Simultaneous parameter tying and sparsification in deep learning. In *International conference on learning representations*.

Zhu, M., & Gupta, S. (2017). To prune, or not to prune: Exploring the efficacy of pruning for model compression. arXiv preprint arXiv:1710.01878.

Zimmer, M., Spiegel, C., & Pokutta, S. (2023). How I learned to stop worrying and Love retraining. In *The eleventh international conference on learning representations*.